Assignment 2: Why Learn Linear Attention?

GStar Bootcamp

Abstract

Long contexts make memory a central design constraint. This assignment follows the path from recurrent matrix memory to the kernels behind Gated DeltaNet and Kimi Delta Attention.

Next: Problem 1. Companion: Implementation reference.

When a model keeps reading

Imagine an AI coding assistant working through a large repository. It reads the project instructions, opens several source files, studies a failing test, and follows a chain of tool results. By the time it starts fixing the bug, the useful context is much larger than the original question. A research assistant faces the same problem with papers and notes; a long conversation creates it one message at a time. We want the model to keep using that history without making every next token increasingly expensive.

Dense causal attention offers a direct way to use the past: a new query can compare itself with the keys of earlier tokens and read their values. During generation, those keys and values are retained in a KV cache. As the history grows, the cache grows with it. A longer context therefore consumes more memory, and producing the next token requires reading more past information. Across many layers and concurrent requests, the history becomes a substantial part of the serving cost.

Assignment 1 addressed one part of this problem. FlashAttention computes softmax attention efficiently by tiling the work and avoiding a materialized full attention matrix. That is a major improvement, but exact dense attention still compares token pairs. Its attention work over a long prompt grows quadratically with prompt length, and its generation cache still grows with the number of retained tokens. Efficiently computing dense attention leaves a second question open: can we change how a layer remembers the past?

A memory that does not grow with the conversation

Linear attention starts from a different representation of history. Instead of retaining a separate key and value for every earlier token in a recurrent layer, it updates a matrix memory. Each token writes into that memory; its query reads the updated result. The next token receives the resulting state and continues the same process.

For fixed head dimensions, the recurrent state has a fixed size. Extending a conversation does not add another row to a history cache in that layer. Doubling the retained context doubles the token-dependent KV storage of a dense-attention layer, while the recurrent layer keeps the same-sized state. Its attention update and read can also do a fixed amount of work per generated token, independent of how many tokens came before. Over a sequence, this gives linear rather than quadratic attention work when the state dimensions are fixed.

Figure 2 estimates Kimi K3's retained attention history. At 1M tokens, its 24 MLA caches need 27 GiB and its 69 KDA states about 0.40 GiB. Replacing those KDA layers with MLA raises the estimate to 104.6 GiB. Both cases keep MLA compression, isolating KDA's contribution. The hypothetical all-attention model is a storage comparison, not a trained ablation.

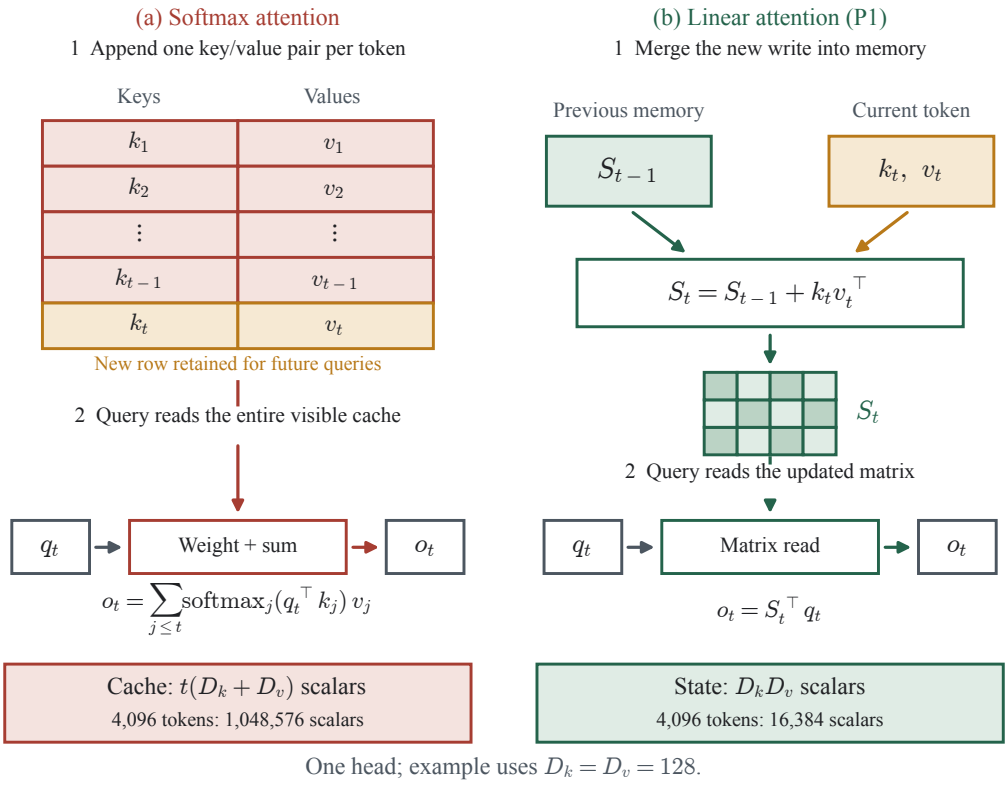


Figure 1. What one attention head stores and reads at token t. (a) Softmax attention appends $(\boldsymbol{k}_t, \boldsymbol{v}_t)$ to its cache, then compares the query with every visible key and combines the corresponding values. (b) P1's linear attention merges the new outer-product write into $\mathbf{S}_t$ and reads that updated matrix; only the matrix is carried forward. With $D_k = D_v = 128$ and 4,096 tokens, these representations contain 1,048,576 versus 16,384 scalars (64 times fewer), excluding weights and temporary buffers. Counts are independent of storage dtype. The fixed state combines associations rather than retaining separate token entries, so the two paths compute different attention operators. KDA later replaces the additive write with gated correction.

This difference becomes more valuable as context grows. Memory released from a growing cache can support longer requests, more concurrent users, or a larger batch on the same hardware. Less history to fetch can also reduce memory traffic during generation. These are architectural benefits, not a promise of the same speedup on every GPU or every workload. Training and prompt processing still need intermediate storage, and a chunked implementation may save boundary states for parallel work. The fixed-size claim concerns the carried recurrent memory during generation, not every allocation made by a complete model.

There is a tradeoff. A fixed matrix cannot preserve an arbitrary token history as separate addressable entries. Linear attention changes the attention mechanism; it is not an exact implementation of softmax attention. The question becomes how to use limited memory well: what should be written, what should be forgotten, and how should an existing association be corrected?

From accumulating information to correcting it

The first memory in this assignment simply adds key–value associations. It is easy to derive and gives us a starting point for a recurrent kernel. But repeated writes can interfere: old information remains in the sum, even when a newer observation should replace it.

Gating introduces forgetting. Before a new association is written, a gate decides how much of the existing memory survives. DeltaNet introduces a complementary idea: first ask what the memory already predicts for the current key, then write a correction toward the current value. A repeated

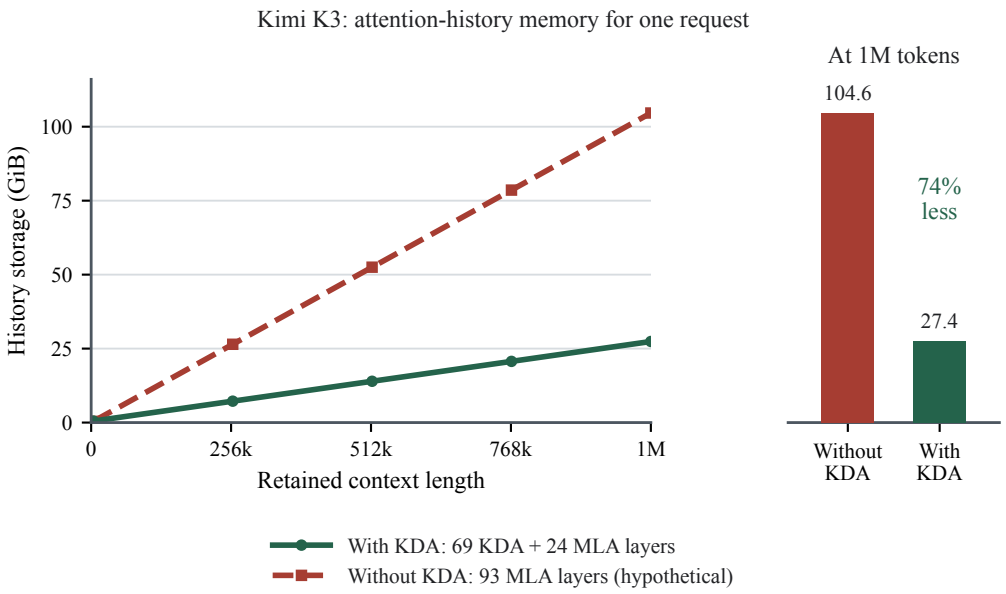


Figure 2. Kimi K3 with and without KDA: calculated logical attention-history storage for one request, before device sharding, using the published layer split [6] and model configuration. "Without KDA" replaces the 69 KDA layers with the same compressed-cache MLA, giving 93 full-attention layers. Assume BF16 MLA cache and FP32 KDA states. At 1M (2^{20}) tokens, the estimates are 27.4 versus 104.6 GiB, about 74% less. These are storage estimates, not measured VRAM; weights, temporary buffers, convolution states and extra prefix checkpoints are excluded.

observation can now revise an association instead of merely adding another copy of it. Gated DeltaNet combines forgetting with this error-correcting update. Kimi Delta Attention makes forgetting more selective by giving different key channels their own gates [1–4].

These steps form the central story of the assignment. We begin with a memory that accumulates, teach it to forget, teach it to correct its predictions, and finally let different parts of that memory forget at different rates. Each change is small enough to understand through a recurrence. Together they lead to a useful modern attention operator.

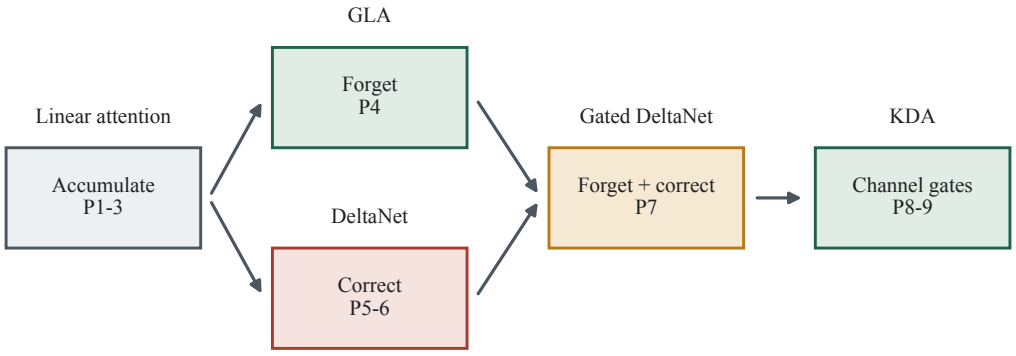


Figure 3. The assignment develops forgetting and correction as separate ideas before combining them. GLA adds forgetting to additive memory; DeltaNet adds error correction. Gated DeltaNet combines both, and KDA makes forgetting channel-wise. Problem numbers identify where you implement each part.

Why these kernels appear in open models

These ideas are already part of important open-weight model architectures. Qwen3-Next combines Gated DeltaNet with gated full-attention layers [5]. Moonshot's Kimi K3 uses KDA together with gated Multi-Head Latent Attention (MLA) [6]. The earlier Kimi Linear report develops the KDA recurrence and its hardware-efficient chunked algorithm [4]. Learning these operators therefore gives you a way to understand architectural choices made in real models, beyond recognizing a name in a model configuration.

The hybrid design matters. Recurrent layers carry a compact summary through much of the network, while attention layers retain a complementary way to access the context. Replacing many growing caches with recurrent states can reduce the model's context-dependent memory substantially. The remaining attention layers still have caches, so the total memory of a hybrid model is not constant in context length. Actual savings depend on the layer mix and cache design. The motivation is to make long-context processing more practical while retaining useful modeling capabilities.

Turning the idea into a GPU implementation

A recurrence explains what the operator computes, but a token-by-token loop alone does not use a GPU well when processing a long prompt. The next step is chunking: express local work as matrix products, pass a compact state between chunks, and compute outputs in parallel once their entry states are known. This is where the mathematical story meets the hardware lessons from Assignment 1.

Problems 1–3 establish that bridge. You first derive matrix memory in PyTorch, implement its recurrent form in Triton, and then reorganize the same computation into chunks. Problem 4 adds channel-wise forgetting. Problems 5–6 introduce delta corrections and the triangular system that lets those corrections be computed within a chunk. Problem 7 combines scalar forgetting with the delta rule. Problem 8 develops the KDA recurrence and a stable gated-score primitive. Problem 9 brings those pieces together into the complete chunked KDA forward pass.

Earlier work becomes part of the final operator. Your KDA implementation reuses your gate scan from Problem 4, triangular inverse from Problem 6, and gated-score builder from Problem 8. You will implement the attention arithmetic rather than call a finished attention operator. By the end, you should be able to explain both why a recurrent layer can avoid a growing token cache and how its chunked kernels reproduce the same state updates.

Start with Problem 1 and follow the problems in order. Read each recurrence, work through a small example, implement one stage, and check it before composing the next. The implementation reference covers layouts and interfaces. Query scaling is omitted from every formula; the code retains `scale`. Use `scale=1.0` to reproduce the equations and worked examples.

References

[1] S. Yang et al. *Gated Linear Attention Transformers with Hardware-Efficient Training*. 2023/2024. https://arxiv.org/abs/2312.06635

[2] S. Yang et al. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. 2024. https://arxiv.org/abs/2406.06484

[3] S. Yang, J. Kautz, and A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. 2024/2025. https://arxiv.org/abs/2412.06464

[4] Kimi Team. *Kimi Linear: An Expressive, Efficient Attention Architecture*. 2025. https://arxiv.org/abs/2510.26692

[5] Qwen Team. *Qwen3-Next: Towards Ultimate Training & Inference Efficiency*. 2025. https://qwen.ai/blog?id=qwen3-next

[6] Moonshot AI. *Kimi K3: Model Introduction and Architecture*. 2026. https://github.com/MoonshotAI/Kimi-K3